Modern Architectures of CNNs

George Michailidis

gmichail@ucla.edu

STAT 426

Many of the building blocks presented in Section 3 emerged in the CNN architectures discussed next.

Introduction to LeNet

- Origins: Proposed by Yann LeCun et al. in the 1990s (AT&T Bell Labs).
- Purpose: Designed to recognize handwritten digits (MNIST dataset) for reading zip codes on mail and numbers on checks.
- Significance:
  - It was the first successful deployment of Convolutional Neural Networks (CNNs).
  - It demonstrated that neural networks could learn features directly from raw pixels, replacing hand-engineered feature extraction.
- The specific architecture discussed is formally known as LeNet-5.

Motivation: Why CNNs over MLPs?

The Problem with Multilayer Perceptrons (MLPs):

- MLPs require flattening images into 1D vectors ($28 \times 28 \rightarrow 784$)
  The strategy in Homework 1.
- Loss of Spatial Structure: Adjacent pixels lose their spatial relationship.
- Parameter Explosion: Fully connecting every pixel to every hidden neuron is computationally expensive (does not scale to images with more complex patterns).

The LeNet Solution:

- Local Interactions: Convolutional layers focus on small regions (kernels).
- Translation Invariance: A feature learned in one corner is recognizable anywhere.
- Parameter Sharing: The same kernel weights are used across the entire image.

LeNet Architecture Overview

LeNet consists of two distinct structural blocks:

1. Convolutional Encoder:
   - Extracts spatial features.
   - Consists of two alternating blocks of Convolutional layers and Pooling layers.
2. Dense Block (Classifier):
   - Compiles features to make a prediction.
   - Consists of three Fully Connected layers.

Data Flow:

Input $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ Conv $\rightarrow$ Pool $\rightarrow$ Flatten $\rightarrow$ FC $\rightarrow$ FC $\rightarrow$ Output

The Convolutional Block Details

- Input: 28×28 single-channel (greyscale).
- Layer 1 (C1 & S2):
  - Conv: 6 output channels, 5×5 kernel, padding=2. (Output: 28×28)
  - Activation: Sigmoid (Historically used; modern nets use ReLU).
  - Pooling: Average Pooling, 2×2, stride 2. (Output: 14×14)
- Layer 2 (C3 & S4):
  - Conv: 16 output channels, 5×5 kernel, no padding. (Output: 10×10)
  - Pooling: Average Pooling, 2×2, stride 2. (Output: 5×5)

The Dense Block (Classifier)

- Flattening:
  - The output of the last pooling layer (16 channels $\times$ 5 $\times$ 5) is flattened into a 1D vector of size 400.
- Fully Connected Layers:
  1. FC 1: Input 400 $\rightarrow$ Output 120 (Sigmoid).
  2. FC 2: Input 120 $\rightarrow$ Output 84 (Sigmoid).
  3. Output Layer: Input 84 $\rightarrow$ Output 10 (Softmax/Gaussian).
- Prediction:
  - 10 probabilities corresponding to digits 0 through 9.

Dimensionality Tracking

Assuming a batch size of 1 and Input ($1 \times 28 \times 28$):

Layer Type	Kernel/Stride	Input Shape	Output Shape
Conv2d	5×5, Pad 2	$1 \times 28 \times 28$	$6 \times 28 \times 28$
AvgPool2d	2×2, Stride 2	$6 \times 28 \times 28$	$6 \times 14 \times 14$
Conv2d	5×5, Pad 0	$6 \times 14 \times 14$	$16 \times 10 \times 10$
AvgPool2d	2×2, Stride 2	$16 \times 10 \times 10$	$16 \times 5 \times 5$
Flatten	-	$16 \times 5 \times 5$	400
Linear	-	400	120
Linear	-	120	84
Linear	-	84	10

The Deep Learning Revolution: AlexNet

- Origin: Introduced by Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton in 2012.
- The Challenge: The ImageNet Large Scale Visual Recognition Challenge.
- Significance:
  - While LeNet (1995) worked on small digits (MNIST), it failed on realistic, high-resolution photography.
  - AlexNet achieved a massive reduction in error rate (almost half of the runner-up), proving that deep CNNs could scale.
- Key Enablers:
  1. Data: ImageNet provided millions of labeled images.
  2. Hardware: The use of GPUs (Graphics Processing Units) made training massive models feasible.

Key Innovations (vs. LeNet)

AlexNet is "evolutionary" (same principles). but introduced critical changes for scalability:

1. ReLU Activation (Rectified Linear Unit):
   - Replaced the Sigmoid/Tanh functions used in LeNet.
   - $f(x) = \max(0, x)$.
   - Benefit: Prevents the vanishing gradient problem and accelerates training.
2. Dropout Regularization:
   - Applied in the fully connected layers.
   - Randomly zeroes out neurons during training.
   - Benefit: Drastically reduces overfitting (crucial for 50M+ parameters).
3. Data Augmentation:
   - Artificially expanded the dataset via random flips, crops, and color jitter.

Architecture Overview

AlexNet is significantly deeper and wider than LeNet. It consists of 8 layers:

- 5 Convolutional Layers:
  - Layers 1, 2, and 5 are followed by Max Pooling (LeNet used Average Pooling).
  - Kernels: Starts large (11×11) to capture global shapes, shrinking to 3×3 for fine textures.
- 3 Fully Connected Layers:
  - Two massive hidden layers with 4096 neurons each.
  - Final output layer matches the 1000 classes of ImageNet.

The Convolutional Layers (Detail)

- Layer 1:
  - Input: $224 \times 224 \times 3$ (RGB Images).
  - Conv: 11×11 kernel, stride 4, 96 output channels.
  - *Note:* Large stride drastically reduces spatial dimension early on.
- Layer 2:
  - Conv: 5×5 kernel, padding 2, 256 output channels.
  - Followed by Max Pooling (3×3, stride 2).
- Layers 3, 4, 5:
  - Stacked directly without pooling in between (to build complex features).
  - L3 & L4: 384 output channels.
  - L5: 256 channels, followed by final Max Pool.

Dimensionality Tracking

Assuming Input Image of size 224×224:

Layer	Kernel / Stride / Pad	Output Channels	Output Shape
Input	-	3 (RGB)	224×224
Conv 1	11×11, S:4, P:1	96	54×54
MaxPool 1	3×3, S:2	96	26×26
Conv 2	5×5, S:1, P:2	256	26×26
MaxPool 2	3×3, S:2	256	12×12
Conv 3	3×3, S:1, P:1	384	12×12
Conv 4	3×3, S:1, P:1	384	12×12
Conv 5	3×3, S:1, P:1	256	12×12
MaxPool 3	3×3, S:2	256	5×5
Flatten	-	-	$6,400$
FC 1	Dropout (0.5)	-	$4,096$
FC 2	Dropout (0.5)	-	$4,096$
Output	Softmax	-	$1,000$

Summary: From LeNet to AlexNet

Feature	LeNet (1995)	AlexNet (2012)
Input Size	28×28 (Greyscale)	224×224 (Color)
Depth	5 Layers	8 Layers
First Kernel	5×5	11×11
Activation	Sigmoid	ReLU
Pooling	Average Pooling	Max Pooling
Regularization	Weight Decay	Dropout
Parameters	$\approx$ 60 Thousand	$\approx$ 60 Million

Takeaway: AlexNet took the architectural ideas of LeNet and scaled them up using new hardware (GPUs) and better non-linearities (ReLU) to solve complex real-world vision tasks.

From Ad-Hoc to Design Patterns: Visual Geometry Group

- Origin: Proposed by the Visual Geometry Group (VGG) at Oxford (Simonyan & Zisserman, 2014).
- The Shift:
  - AlexNet showed that deep networks work, but its architecture was somewhat ad-hoc (irregular kernel sizes $11 \times 11, 5 \times 5$).
  - VGG introduced the concept of Blocks: repeating patterns of layers.
- Key Insight:
  - Replace large kernels with stacks of small 3×3 kernels.
  - This created the philosophy of "Deep and Narrow".

Why only 3×3 Convolutions?

VGG replaced the large 5×5 and 7×7 kernels with stacks of 3×3 kernels. Why?

1. Same Receptive Field:
   - Two stacked 3×3 layers see the same amount of pixels as one 5×5 layer.
   - Three stacked 3×3 layers see the same as one 7×7 layer.
2. Fewer Parameters:
   - 5×5 kernel params: **25**
   - Two 3×3 kernels params: $3^2 + 3^2 = 9 + 9 = \mathbf{18}$
   - Result: We can make the network deeper for the same cost.
3. More Non-Linearity:
   - Two layers mean two ReLU functions instead of one, allowing the network to learn more complex features.

3. The Fundamental Building Block

The architecture is built entirely by repeating this specific VGG Block:

VGG Block Structure

1. Convolution Layers:
   - Kernel Size: 3×3
   - Padding: 1 (Maintains height/width)
   - Stride: 1
   - Activation: ReLU
2. Max Pooling Layer:
   - Kernel Size: 2×2
   - Stride: 2 (Halves height/width)

Design Pattern: The spatial dimension shrinks ($224 \to 112 \to 56 \ldots$) while the depth (output channels) increases ($64 \to 128 \to 256 \ldots$).

VGG-11 Architecture

The "VGG-11" (8 Conv + 3 FC layers) is composed of 5 Blocks:

1. Block 1: 1 Conv layer, 64 Channels.
2. Block 2: 1 Conv layer, 128 Channels.
3. Block 3: 2 Conv layers, 256 Channels.
4. Block 4: 2 Conv layers, 512 Channels.
5. Block 5: 2 Conv layers, 512 Channels.

After the blocks, the features are flattened and passed through:

$$\text{Flatten} \rightarrow \text{FC}(4096) \rightarrow \text{FC}(4096) \rightarrow \text{Output}(1000)$$

5. Dimensionality Tracking (VGG-11)

Assuming Input Image of size 224×224:

Block	Convolutions	Channels	Output Shape
Input	-	3	224×224
Block 1	1 layer	64	112×112 (after pool)
Block 2	1 layer	128	56×56 (after pool)
Block 3	2 layers	256	28×28 (after pool)
Block 4	2 layers	512	14×14 (after pool)
Block 5	2 layers	512	7×7 (after pool)
Flatten	-	-	$25,088$ $(512 \times 7 \times 7)$
FC 1	Dropout (0.5)	-	$4,096$
FC 2	Dropout (0.5)	-	$4,096$
Output	Softmax	-	$1,000$

Summary & Trade-offs

Strengths:

- Simplicity: Very easy to implement using loops (repeated blocks).
- Generalization: VGG features transfer very well to other tasks (object detection, segmentation).

Weaknesses:

- Computationally Expensive: VGG-19 is extremely slow compared to modern networks like ResNet.
- Memory Heavy:
  - The first FC layer connects $25,088$ inputs to $4,096$ outputs.
  - That alone is $\approx$ **102** Million parameters.
  - VGG-16 total size is over 500MB.

Network in Network (NiN) — Overview

- Limit of Classic CNNs: They use linear filters followed by a simple nonlinearity. This might be too simple to extract complex features from a local patch.
- The NiN Idea: Embed small neural networks (micro MLPs) inside convolutional layers to make local feature extraction more expressive.
- Implementation: These micro-networks are implemented via 1×1 convolutions.
- Structural Shift: Replaces the "memory-hungry" fully connected layers at the end with Global Average Pooling (GAP).

Motivation: Why 1×1 Convolutions?

- Classic Conv: One kernel produces one feature at a time ($Input \cdot Kernel + Bias$). It is a linear local operator.
- NiN Block: Treats each spatial pixel across all channels as a vector input to a shared MLP.
- The Equivalence:
  - A 1×1 convolution with C_{out} filters is mathematically identical to a **Fully Connected (Dense) layer** applied to each pixel independently.
  - Parameters: $C_{in} \times C_{out}$.
- Benefit: It allows complex, nonlinear mixing of channel information without changing the spatial dimensions ($H \times W$).

The NiN Block: A "Mini-Network" for Every Pixel

A NiN block replaces the standard Convolution layer with a tiny **MLP** that slides across the image.

The Block Structure:

1. **Spatial Layer:** 3×3 Conv (Extracts neighbor info)
2. **MLP Layer 1:** 1×1 Conv + ReLU (Feature mixing)
3. **MLP Layer 2:** 1×1 Conv + ReLU (Further abstraction)

The Secret

The 1×1 Conv is just a **Matrix Multiplication** performed on the channel vector at every pixel (x, y).

Step 1: The Spatial Foundation

Input: $28 \times 28 \times 160$

Operation: 3×3 Convolution

- Kernels: 160 filters of size $3 \times 3 \times 160$ - assume padding=1, stride=1.
- Math: $Y_{i,j} = \text{ReLU}(\text{Weights} * \text{Input Window} + \text{bias})$
- Result: A $28 \times 28 \times 160$ map where each pixel "sees" its 3×3 neighborhood.

This layer finds local patterns (e.g., "is there a vertical edge here?").

Step 2: The First Local MLP Layer

Operation: 1×1 Convolution (The Linear Transformation)

- At every pixel (x, y), we have a vector $\mathbf{v} \in \mathbb{R}^{160}$.
- We multiply it by a Weight Matrix $W \in \mathbb{R}^{160 \times 160}$.

The ReLU Operator (The Non-Linear Decision):

- Equation: $Z_{x,y} = \max(0, W\mathbf{v} + b)$
- Dimensions: $28 \times 28 \times 160 \rightarrow \mathbf{28} \times \mathbf{28} \times \mathbf{160}$

This is a Hidden Layer. It does not look at neighbors anymore; it just asks: "Based on the 160 patterns found in Step 1, does this pixel look like part of a 'Dog's Ear'?"

Step 3: The Second Local MLP Layer

Operation: Another 1×1 Convolution + ReLU

- Input: Output from Step 2 ($28 \times 28 \times 160$).
- Math: $A_{x,y} = \max(0, W_2 Z_{x,y} + b_2)$

- Why two 1x1 layers?
  1. Without the ReLU in Step 2, two 1×1 convs would collapse into one single linear layer.
  2. With ReLU, we have a Deep MLP that can model extremely complex per-pixel logic.

This is the Output Layer of the "Mini-Brain" sitting on this pixel.

The Global Transition: From 160 to 10

How do we get from $28 \times 28 \times 160$ to our final $7 \times 7 \times 10$ input?

1. Spatial Shrinkage

- Between NiN blocks, we apply MaxPooling.
- Two rounds of 2×2 pooling (stride 2) shrink the grid:

$$28 \times 28 \rightarrow 14 \times 14 \rightarrow \mathbf{7} \times \mathbf{7}$$

Channel Evolution (The Features)

- Early Blocks: Use 160 filters to find complex textures/parts.
- The Final Block: Is configured with **10 filters**.
- Its 1×1 convs act as a per-pixel classifier: "Based on these 160 features, does this specific pixel belong to Class 1, Class 2, ..., or Class 10?"

Result

We end up with 10 "Heatmaps" of size 7×7. Each map represents the confidence for one specific class across the image grid.

Step 4: Global Average Pooling (GAP)

Based on pooling the final block has **Classes** (e.g., 10).

- Input: $7 \times 7 \times 10$ (10 feature maps, one for each class).
- Operation: Average all pixels in each 7×7 map.
- Math: $\text{Score}_c = \frac{1}{49} \sum_{i=1}^{7} \sum_{j=1}^{7} \text{Pixel}_{i,j,c}$

Instead of a Fully Connected layer at the end (like AlexNet/VGG), NiN just asks: "On average, how much 'Dog' signal did we find across the whole image?"

NiN Dimension Cheat Sheet: From Pixels to Probabilities

Stage	Operation	Input Shape	Output Shape	Feature Meaning
Input	Raw Image	–	$28 \times 28 \times 3$	RGB Pixels
Block 1	3×3 Conv + 1×1s	$28 \times 28 \times 3$	$28 \times 28 \times 160$	Edges / Colors
Down	MaxPool (2×2)	$28 \times 28 \times 160$	$14 \times 14 \times 160$	Resolution $\downarrow$
Block 2	3×3 Conv + 1×1s	$14 \times 14 \times 160$	$14 \times 14 \times 160$	Shapes / Parts
Down	MaxPool (2×2)	$14 \times 14 \times 160$	$7 \times 7 \times 160$	Spatial Grid $\downarrow$
Final Block	1×1 Conv (10 filters)	$7 \times 7 \times 160$	$7 \times 7 \times 10$	**Class Heatmaps**
Global Pool	Global Avg Pool	$7 \times 7 \times 10$	$1 \times 1 \times 10$	**Final Scores**

Transformation Logic

- **Early/Mid Stages:** Increase **depth** (number of channels) to learn rich features.
- **Final Stage:** Reduce depth to the number of classes (10), then collapse spatial dimensions ($7 \times 7 \rightarrow 1 \times 1$) using Global Average Pooling.

Parameter Efficiency: NiN vs. Traditional FC

Let's compare the Parameter Count for the classification head ($C_{in} = 160$, Classes $= 10$):

Scenario A: AlexNet/VGG Style (Flattening)

- **Input:** $7 \times 7 \times 160$ feature maps.
- **Flattening:** Creates a 1D vector of 7840 units.
- **FC Layer:** Connects $7840 \rightarrow 10$ output units.
- Parameters: $7840 \times 10 + 10 = \mathbf{78,410}$

Scenario B: NiN Style (GAP)

- **Final** 1×1 **Conv:** Reduces 160 channels $\rightarrow$ 10 channels.
- Parameters: $160 \times 10 + 10 = \mathbf{1,610}$
- **GAP Step:** Averages each 7×7 map ($7 \times 7 \rightarrow 1 \times 1$).
- GAP Parameters: **0** (it is a fixed math operation).

Summary

NiN provides a $\approx$**50x reduction** in parameters. By using GAP instead of Flattening, we force the network to learn **global concepts** rather than **specific pixel positions**.

Summary: Classic CNN vs. NiN

Feature	Classic (VGG)	NiN
Local Operator	Linear Filter + ReLU	Micro-MLP (1x1 Convs)
Final Layers	Dense (FC) Layers	Global Average Pooling
Parameters	Very High (in FC)	Significantly Lower
Overfitting Risk	High	Low (Regularized by GAP)

The Legacy of NiN:

- 1×1 convolutions are the "glue" of Inception and ResNet.
- GAP is now the standard for modern classification heads.

Motivation: Why 1×1 Convolutions?

- **Classic Conv:** A sliding window that acts as a linear local operator.
- **The Problem:** Classic convs are great at spatial features but weak at "channel-wise" reasoning.
- **The NiN Solution:** Treat each spatial pixel across all channels as a vector input to a **shared MLP**.
- **The Equivalence:**
  - A 1×1 convolution with C_{out} filters is mathematically identical to a **Fully Connected (Dense) layer** applied to each pixel independently.
  - **Dimensions:** It transforms depth ($C_{in} \rightarrow C_{out}$) while preserving height and width ($H \times W$).

The NiN Block: A "Mini-Brain" for Every Pixel

A NiN block replaces the standard Convolution layer with a tiny **Multi-Layer Perceptron (MLP)** that slides across the image.

The Block Architecture:

1. **Spatial Layer:** 3×3 Conv (Extracts neighbor info)
2. **MLP Layer 1:** 1×1 Conv + ReLU (First hidden layer)
3. **MLP Layer 2:** 1×1 Conv + ReLU (Second hidden layer)

Insight for Students

The 3×3 layer "looks around" to see local context. The 1×1 layers "think" about what those combined features mean at that specific spot.

Step 1: The Spatial Foundation

Input Shape: $28 \times 28 \times 160$ (Height $\times$ Width $\times$ Channels)

Operation: 3×3 **Convolution**

- **Settings:** 160 filters, **padding=1**, stride=1.
- **Math:** $Y = \text{ReLU}(\text{Weight} * \text{Input} + b)$
- **Output Shape:** $28 \times 28 \times 160$

Pedagogical Note: Because we used **padding=1**, the spatial resolution (28×28) is preserved. We are just "sharpening" our features before the MLP kicks in.

Step 2: The First Local MLP Layer

Operation: 1×1 **Convolution + ReLU**

- **Input:** A vector $\mathbf{v} \in \mathbb{R}^{160}$ at every pixel (x, y).
- **Linear Step:** Multiply by Weight Matrix $W_1 \in \mathbb{R}^{160 \times 160}$.
- **Activation:** $Z_{x,y} = \max(0, W_1\mathbf{v} + b_1)$

Resulting Shape: $28 \times 28 \times 160$

Student View: This is exactly like a Hidden Layer in a standard MLP. It allows the network to learn complex correlations **between** the 160 different feature maps.

Step 3: Deepening the Logic

Operation: Second 1×1 Convolution + ReLU

- **Input:** Output from Step 2 ($28 \times 28 \times 160$).
- **Activation:** $A_{x,y} = \max(0, W_2 Z_{x,y} + b_2)$

Why two layers?

- One 1×1 conv is just a linear projection.
- Adding a second 1×1 layer with a ReLU in between creates a **Deep MLP**. This "Mini-Brain" can now solve much more non-linear problems at each pixel.

The Transition: From Features to Classes

A full NiN model stacks several of these blocks. Two things happen as we move toward the output:

1. **Spatial Shrinkage (The Resolution)**
   - Between NiN blocks, we apply **MaxPooling** (2×2, stride 2).
   - $28 \times 28 \rightarrow 14 \times 14 \rightarrow \mathbf{7} \times \mathbf{7}$

2. **Channel Evolution (The Final Block)**
   - **The Special Case:** In the very last NiN block, we set $C_{out} =$ **Classes** (e.g., 10).
   - Its 1×1 convs act as a per-pixel classifier.

Final Shape before Output: $7 \times 7 \times 10$

Step 4: Global Average Pooling (GAP)

In NiN, we don't "Flatten." We average.

- **Input:** $7 \times 7 \times 10$ (One "heatmap" per class).
- **Operation:** Average all 49 pixels in each map.
- **Math:** $\text{Score}_c = \frac{1}{49} \sum_{i=1}^{7} \sum_{j=1}^{7} \text{Pixel}_{i,j,c}$

Philosophy Change

Instead of learning a specific position (e.g., "The dog is in the top left"), GAP asks: **"How much total 'Dog-ness' is present in the entire image?"**

Parameter Efficiency: NiN vs. Traditional FC

Let's compare the Parameter Count for the classification head ($C_{in} = 160$, Classes $= 10$):

Scenario A: AlexNet/VGG Style (Flattening)

- Input: $7 \times 7 \times 160$ map.
- Flatten to vector of size **7,840**.
- FC Layer: $7,840 \times 10$ weights $+ 10$ biases.
- Parameters: **78,410**

Scenario B: NiN Style (1x1 Conv + GAP)

- Last 1×1 Conv: 160 channels $\rightarrow$ 10 channels.
- Parameters: $(1 \times 1 \times 160) \times 10 + 10 = \mathbf{1,610}$.
- GAP Step: **0 parameters**.

Summary

NiN is roughly **50x more efficient** in the classification head, significantly reducing overfitting.

Motivation: Moving Beyond VGG

By 2014, the consensus was: Deeper is Better. But how do we go deeper without the "Computational Explosion"?

- Key Question: Should we use $1 \times 1, 3 \times 3$, or 5×5 filters?
  - Small kernels ($1 \times 1, 3 \times 3$) capture fine details.
  - Large kernels (5×5) capture global, sparse features.
- GoogLeNet's Solution: Don't choose. Use them all in parallel.

Key Innovation:
The Inception Block acts as a "multi-scale" feature extractor, allowing the network to decide which size filter is most important for a given feature.

Anatomy of an Inception Block

An Inception block consists of four parallel paths that are concatenated at the end.

1. **Path 1:** 1×1 Convolution.
2. **Path 2:** 1×1 Conv $\rightarrow$ 3×3 Conv.
3. **Path 3:** 1×1 Conv $\rightarrow$ 5×5 Conv.
4. **Path 4:** 3×3 Max Pool $\rightarrow$ 1×1 Conv.

Note: All paths use padding to ensure the height and width of the output remain identical for concatenation.

The 1x1 "Bottleneck" Strategy

Why are there 1×1 convolutions *before* the 3×3 and 5×5 layers?

The Computational Challenge:

- 5×5 convolutions are expensive. If the input has 192 channels, the parameter count is massive.

The Solution (Dimensionality Reduction):

- We use a 1×1 convolution to "squish" 192 channels down to (e.g.) 16 channels.
- The 5×5 convolution then processes only 16 channels.

Parameter Savings

By adding these "bottleneck" 1×1 layers, GoogLeNet achieves a **22-layer depth** with only **7 million parameters** (AlexNet had 60 million!).

GoogLeNet: The Full Pipeline

GoogLeNet is organized into 5 stages (or modules):

- **Stage 1 & 2:** Standard Conv and Max-Pooling (The "Stem").
- **Stage 3, 4 & 5:** Stacks of Inception Blocks.
- **The Head:** Like NiN, it uses **Global Average Pooling (GAP)** instead of heavy FC layers.

Digression: the Calculations for the "Squish": 1x1 Convolution

How do we reduce 192 channels to 16? Recall that we treat each pixel in the feature map as a vector (since we have in this example 192 such maps/channels).

1. The Parameter Count Definition For a layer with C_{in} input channels and C_{out} output channels:

$$\text{Params} = (K_h \times K_w \times C_{in}) \times C_{out} + \text{biases}$$

For our "Squish" (1×1 Conv):

- $K_h = 1, K_w = 1$ (the spatial footprint is a single pixel).
- $C_{in} = 192, C_{out} = 16$.
- Math: $(1 \times 1 \times 192) \times 16 = \mathbf{3,072}$ weights.

The Matrix Equivalence

At every spatial location (i, j), we perform a matrix-vector multiplication:

$$\mathbf{y}_{i,j} = \sigma(W\mathbf{x}_{i,j} + \mathbf{b})$$

Where $\mathbf{x} \in \mathbb{R}^{192}$, $W \in \mathbb{R}^{16\times192}$, and $\mathbf{y} \in \mathbb{R}^{16}$.

Why Bottlenecks? A Parameter Comparison

Goal: Apply a 5×5 convolution to 192 channels to get 32 output channels.

Option A: Direct 5x5 Convolution

- Math: $(5 \times 5 \times 192) \times 16 = \mathbf{76,800}$ parameters.
- *Cost:* Very heavy; the kernel must process all 192 channels at once.

Option B: 1x1 Bottleneck $\rightarrow$ 5x5 Convolution

1. **Reduce:** 1×1 conv $(192 \rightarrow 16)$: $(1 \times 1 \times 192) \times 16 = \mathbf{3,072}$
2. **Process:** 5×5 conv $(16 \rightarrow 16)$: $(5 \times 5 \times 16) \times 16 = \mathbf{6,400}$

The Efficiency Win

Total Params (Option B): $3,072 + 6,400 = \mathbf{9,472}$
Savings: $\approx 8\times$ fewer parameters for the same spatial result!

Insight: The 1×1 layer selects the most useful features before the expensive 5×5 layer does the spatial work.

Summary: Why GoogLeNet Won ILSVRC 2014

Feature	Impact
Parallel Paths	Multi-scale feature extraction.
1x1 Bottlenecks	Reduced complexity and parameter count.
Deep & Sparse	Efficient use of computing resources.
No Large FC Tail	Massive reduction in overfitting risk.

Legacy:

- The 1×1 bottleneck became a standard design pattern in modern networks like **ResNet** and **DenseNet**.
- Proved that **structural sparsity** is superior to brute-force depth.

Motivation: Why Normalize?

- Data Preprocessing: Standardizing input features ($\mu = 0, \sigma = 1$) makes optimization much easier.
- The Internal Challenge: As a network trains, weights in early layers change. This shifts the distribution of inputs to later layers (*Internal Covariate Shift*).
- Numerical Stability: Intermediate layer outputs can take widely varying magnitudes, leading to:
  - Vanishing gradients (e.g., in Sigmoid/Tanh).
  - Exploding gradients.
  - Slow convergence.

Batch Normalization (Ioffe and Szegedy, 2015)

Apply a normalization step inside the network for every mini-batch.

The Mathematical Formulation

For a mini-batch $\mathcal{B} = \{x_1, \ldots, x_m\}$, the BatchNorm transform $BN_{\gamma,\beta}(x_i)$ is:

1. **Batch Mean:** $\mu_{\mathcal{B}} = \frac{1}{m}\sum_{i=1}^{m} x_i$
2. **Batch Variance:** $\sigma_{\mathcal{B}}^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2 + \epsilon$
3. **Normalize:** $\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sigma_{\mathcal{B}}}$
4. **Scale and Shift:** $y_i = \gamma \odot \hat{x}_i + \beta$

- γ (scale) and β (shift) are **learnable parameters**.
- $\epsilon > 0$ is a small constant to prevent division by zero.
- BatchNorm allows the network to recover the original distribution if needed ($\gamma = \sigma, \beta = \mu$).

Placement in the Network

BatchNorm is typically applied after the affine transformation (Conv or Fully Connected) and before the activation function.

1. Fully Connected Layers:
   - $h = \phi(\mathrm{BN}(Wx + b))$
   - Statistics are calculated across all examples in the batch for each feature.
2. Convolutional Layers:
   - Statistics are calculated **per channel**, across all examples *and* all spatial locations ($m \cdot p \cdot q$ elements).
   - This preserves translation invariance: the same γ and β are applied to all pixels in a channel.

Prediction Mode (Inference)

During testing/prediction, we might process only **one example** at a time. A single example has no meaningful batch variance.

Solution: Moving Averages

- During training, the layer maintains a **running mean** and **running variance**.
- These are updated using momentum (typically 0.9 or 0.99):
- Running Mean $\leftarrow (1 - \text{momentum}) \cdot \text{Running Mean} + \text{momentum} \cdot \mu_{\mathcal{B}}$

Key Result

At test time, the BatchNorm behavior is **deterministic** and depends only on the statistics accumulated during training.

Why Use BatchNorm?

- **Faster Convergence:** Allows for significantly higher learning rates.
- **Less Sensitivity:** The network is less sensitive to parameter initialization.
- **Regularization Effect:** Because statistics are calculated over mini-batches, it adds slight noise to the activations, acting as a form of regularization (similar to Dropout).

Implementation Tip

If using BatchNorm, the bias term (b) in the preceding Conv/FC layer is redundant because the β parameter in BatchNorm performs the same shifting role.

The Challenge of Going Deeper

- Intuition: Adding more layers should increase a network's expressive power.
- Challenge: Simply stacking layers often leads to *degradation*: higher training error as depth increases.
- Key Issue: How do we ensure that adding a layer makes a network at least as good as its shallower version?

Function Classes

If a larger function class $\mathcal{F}$ contains the smaller one $\mathcal{F}'$, we are guaranteed that increasing complexity doesn't move us away from the optimal solution.

From Identity to Residuals

If the identity mapping $f(x) = x$ is optimal, it is easier to push a layer to learn $g(x) = 0$ than to learn the identity function from scratch.

Regular Block: Attempts to learn the mapping $f(x)$ directly.

Residual Block: Learns the **residual mapping** $g(x) = f(x) - x$. The final output is $g(x) + x$.

- The connection that passes x directly to the addition is called a **residual connection** (or shortcut).
- This allows gradients to propagate more easily through deep layers.

Key Elements of a Residual Block

As a working example, consider a standard residual block based on VGG's 3×3 design:

1. Path A (Weight Path):
   - 3×3 Convolution
   - Batch Normalization
   - ReLU Activation
   - 3×3 Convolution
   - Batch Normalization
2. Path B (Shortcut Path):
   - Identity mapping (if shapes match).
   - 1×1 Convolution (if we need to change channels or stride).

Final Step: Add Path A + Path B, then apply the final ReLU.

Handling Dimension Mismatches

The operation $g(x) + x$ requires $g(x)$ and x to have the same shape.

- **Case 1: Same Shape.** x is added directly.
- **Case 2: Different Shape.** When we double the channels or halve the resolution (stride=2):
  - We use a 1×1 **convolution** in the shortcut path.
  - This adjusts x to match the output dimensions of the weight path.

The Full ResNet-18 Model

ResNet-18 is composed of 5 stages:

Stage	Structure	Output Size
Stem	7×7 Conv, Stride 2, MaxPool	$56 \times 56 \times 64$
Res-Block 1	2 Residual Blocks (64 channels)	$56 \times 56 \times 64$
Res-Block 2	2 Residual Blocks (128 channels)	$28 \times 28 \times 128$
Res-Block 3	2 Residual Blocks (256 channels)	$14 \times 14 \times 256$
Res-Block 4	2 Residual Blocks (512 channels)	$7 \times 7 \times 512$
Head	Global Avg Pool, Fully Connected	10 (or 1000)

Note: The first block of each stage (except the first) downsamples the input.

Why ResNet Had a Big Impact

- Training Stability: Allows training of networks with hundreds or thousands of layers.
- Gradient Flow: Shortcuts act as "highways" for the gradient, mitigating vanishing gradient problems.
- Legacy: Residual connections are now a standard component in nearly all modern architectures (including Transformers/BERT/GPT).

From ResNet to DenseNet

Both architectures use "shortcuts" to improve gradient flow, but they treat the information differently:

- ResNet (Summation):

$$x_\ell = \mathcal{F}(x_{\ell-1}) + x_{\ell-1}$$

  - The learned residual $\mathcal{F}$ and the identity are added.
  - This creates a "highway" for gradients but combines features into a single value.

- DenseNet (Concatenation):

$$x_\ell = H_\ell([x_0, x_1, \ldots, x_{\ell-1}])$$

  - Layer ℓ receives feature maps from all preceding layers.
  - Features are concatenated (stacked) along the channel dimension, preserving the original information.

The Dense Block

A Dense Block consists of multiple convolutional layers where each layer's input is the concatenation of all previous layers' outputs.

- Growth Rate (k): If each layer produces k feature maps, then the ℓ-th layer will have $C_0 + k \times (\ell - 1)$ input channels.
- Feature Reuse: This architecture is extremely efficient at reusing features, as early-layer information is directly accessible to the very last layer.

Calculations: The output of layer ℓ is $x_\ell = H_\ell([x_0, x_1, \cdots, x_{\ell-1}])$, where $[\ldots]$ denotes concatenation.

Transition Layers: Managing Complexity

Because concatenation increases the number of channels rapidly, DenseNet uses Transition Layers between Dense Blocks to keep the model size manageable.

Components of a Transition Layer:

1. Batch Normalization
2. 1x1 Convolution: Reduces the number of channels (Compression).
3. 2x2 Average Pooling: Halves the spatial dimensions ($H \times W$).

Key Insight: Dense blocks increase depth and channel count; Transition layers shrink the maps and compress the depth.

Global Architecture

DenseNet follows a similar modular structure to ResNet but swaps Residual Blocks for Dense Blocks.

Component	**Function**
Stem	7×7 Conv + MaxPool (Initial downsampling).
Dense Block 1	Sequence of k convolutions with concatenation.
Transition 1	1×1 Conv + 2×2 AvgPool.
Dense Block 2	Further feature extraction and reuse.
...	...
Classification Head	Global Avg Pool + Fully Connected Layer.

Why DenseNet?

- Strong Gradient Flow: Every layer has a direct "shortcut" to the loss function via the concatenated paths.
- Parameter Efficiency: Because features are reused, DenseNet often requires **fewer parameters** than ResNet for similar accuracy.
- Bottleneck Layers: Modern implementations use 1×1 convs *within* the dense block (DenseNet-B) to reduce input channels before the 3×3 conv.

Accessing Architectures: Hugging Face & Libraries

Modern CNNs are rarely written from scratch in production. Instead, we use Model Hubs and specialized libraries.

- The Hugging Face Hub: A central repository for weights and configurations for thousands of models (ResNet, DenseNet, ConvNeXt).
- timm (PyTorch Image Models): The "gold standard" library integrated with Hugging Face for Computer Vision.

Loading a Model in Python:

- `timm.create_model('resnet50', pretrained=True)`
- `timm.create_model('densenet121', pretrained=True)`